

Deep Learning for NLP(3)

Student name: *Petros Matsoukis*
sdi: sdi2100099

Course: *Artificial Intelligence II*
Semester: *Spring Semester 2025*

Contents

1	Kaggle Notebooks	3
2	BERT model	3
2.1	Abstract	3
2.2	Data processing and analysis for BERT model	3
2.2.1	Pre-processing	3
2.2.2	Data partitioning for train, test and validation for BERT model	3
2.2.3	Vectorization	4
2.3	Experiments and hyper-parameter tuning for BERT model	4
2.3.1	DataLoader	4
2.3.2	BERT Classifier	5
2.3.3	Optimizer	5
2.3.4	Epochs	6
2.3.5	BERT Dataset	7
2.4	Results, plots and Overall Analysis for BERT model	8
2.4.1	ROC curve	8
2.4.2	Learning Curve for loss	8
2.4.3	Confusion matrix	9
2.5	Best trial	10
3	DistilBERT model	11
3.1	Abstract	11
3.2	Data processing and analysis for distilBERT model	11
3.2.1	Pre-processing	11
3.2.2	Data partitioning for train, test and validation for distilBERT model	12
3.2.3	Vectorization	12
3.3	Experiments and hyper-parameter tuning for distilBERT model	13
3.3.1	DistilBERT Dataset	13
3.3.2	DataLoader	13
3.3.3	Optimizer	14
3.3.4	DistilBERT Classifier	16
3.3.5	Scheduler	16
3.4	Results, plots, and Overall Analysis for DistilBERT model	17

3.4.1	ROC Curve	17
3.4.2	Learning Curve for loss	18
3.4.3	Confusion Matrix	19
3.5	Best trial	19
4	Comparison of BERT and distilBERT model	20
5	Comparison with TF-IDF + Logistic Regression model	20
6	Comparison with Word2Vec + Neural Network model	21

1. Kaggle Notebooks

- BERT: [Notebook for BERT model](#)
- DistilBERT: [Notebook for DistilBERT model](#)

2. BERT model

2.1. Abstract

In this project, I tackle the task of sentiment classification on English-language tweets using a pretrained BERT model, implemented with PyTorch and the HuggingFace Transformers library. The dataset is preprocessed and tokenized using BERT's tokenizer, and each tweet is converted into input tensors compatible with the model. I define a custom classifier built on top of the BERT base architecture, fine-tuning it on the training data using the AdamW optimizer and a learning rate scheduler. The model is trained and validated over multiple epochs, with performance tracked using accuracy, precision, recall, and F1-score. I incorporate dropout for regularization and save the model achieving the best validation accuracy. After training, I evaluate the final model using a confusion matrix, ROC curve, and learning curve for loss, and use it to generate sentiment predictions on the test dataset. To ensure reproducibility, random seeds are consistently applied throughout the pipeline.

2.2. Data processing and analysis for BERT model

2.2.1. Pre-processing. BERT does not need text preprocessing because it is already pretrained on large amounts of natural language, including raw text with punctuation, casing, and informal language like tweets. Its tokenizer can handle misspellings, slang, and uncommon words by breaking them into smaller pieces, so we do not need to clean the text too much before using it.

2.2.2. Data partitioning for train, test and validation for BERT model. Dataset was already partitioned into 3 files (train, val and test).

- **Training dataset:** This is the data used to train the model. The model learns from it so that it can predict labels to other datasets later.
- **Validation dataset:** After each epoch, the model is evaluated on the validation set. It is used to tune hyperparameters and generally select the best version of the model.
- **Test dataset:** The model predicts labels on real data that the model has never seen before and gives the final estimate of its true performance.

2.2.3. Vectorization. Vectorization is handled by the BERT tokenizer, which converts each input tweet into a sequence of token IDs. These tokens are numerical representations of words or subwords, based on BERT's vocabulary. The tokenizer also adds special tokens like [CLS] and [SEP], pads or truncates the sequences to a fixed length, and creates attention masks to indicate which parts of the input are actual tokens and which are padding. These token IDs are then passed into the pretrained BERT model, which transforms them into high-dimensional vector representations capturing the meaning and context of the input text. These vectors are then used as input to the classifier for sentiment prediction.

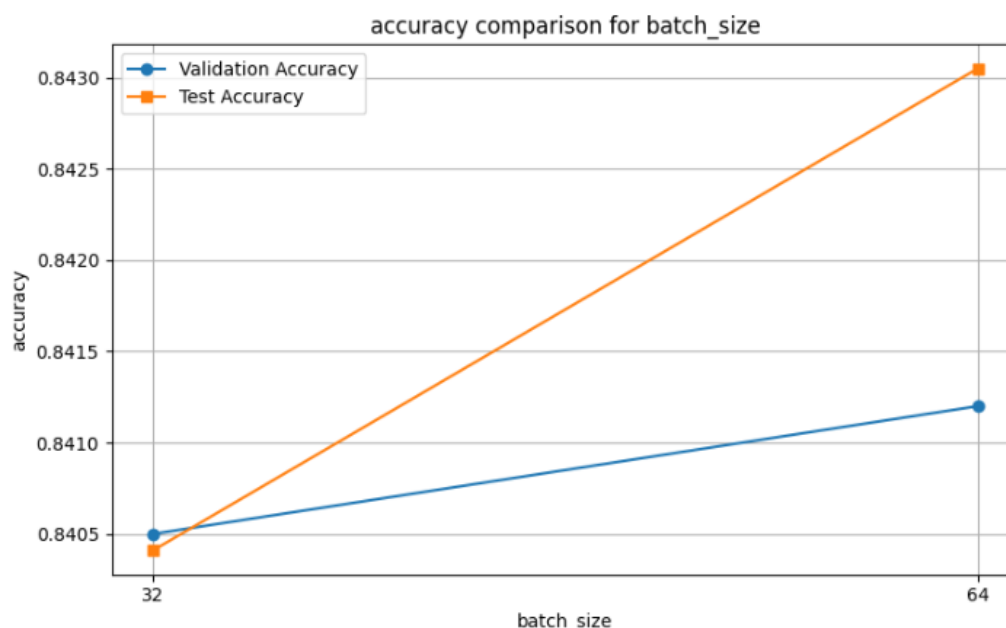
2.3. Experiments and hyper-parameter tuning for BERT model

For the experiments and hyperparameter tuning, Since hyperparameter tuning on the full dataset required a significant amount of time, I created a random subset containing one-fourth of the total tweets. This sample is balanced, which means that it includes an equal number of tweets labeled 0 and 1, as the full dataset already has a nearly equal distribution between the two classes.

2.3.1. DataLoader. The hyperparameter values when the experiments started were: batch_size=64, shuffle=True (only for train dataset) and generator=g (only for train dataset).

- **batch_size:** Batch size is the amount of training examples that the model processes at once (at each step in an epoch) before updating its internal weights to learn from them. The plot below shows how train, val and test accuracy perform when different values are tested (32, 64).

Both val_accuracy and test_accuracy reach their best value with 64 as the plot below shows. For this reason, the best value for batch_size is 64. All the below experiments are performed with batch_size=64.



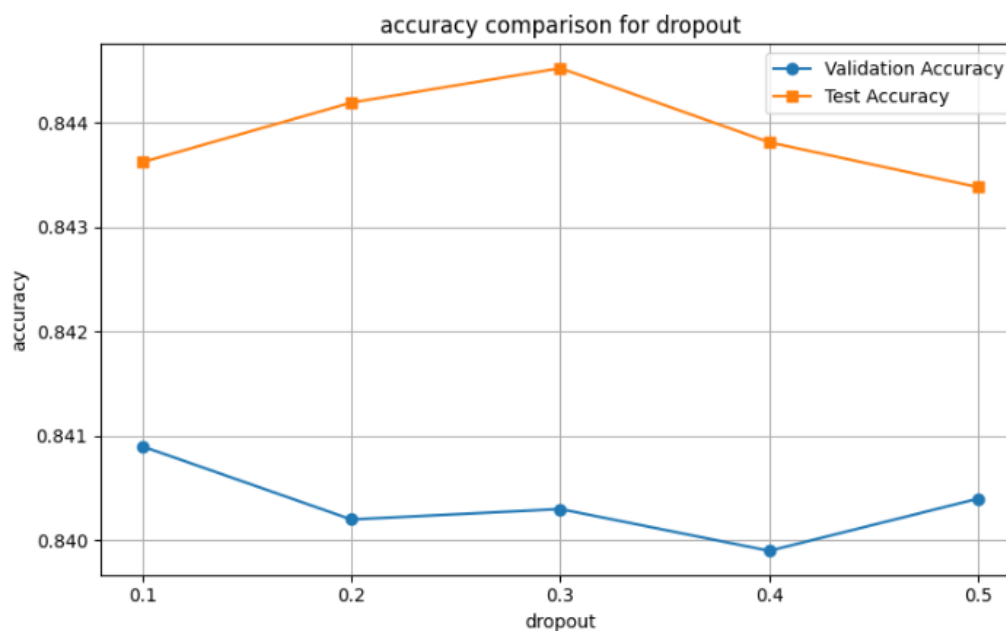
- **shuffle:** For DataLoader of validation dataset, shuffle must be False, because the model will predict labels and the tweets cannot be shuffled. For the train-

ing dataset, shuffle must be True because shuffling improves generalization and avoids overfitting.

- **generator:** By passing a `torch.Generator()` with a fixed seed(42), it is ensured that shuffling is deterministic and reproducible.

2.3.2. BERT Classifier. The BERT classifier is a neural network model that uses the pretrained BERT (Bidirectional Encoder Representations from Transformers) architecture to perform text classification tasks, such as sentiment analysis. It works by first passing input text through BERT, which transforms each sentence into a contextualized representation based on the meaning of its words and their relationships. From BERT's output, the [CLS] token embedding (which represents the whole input) is extracted and passed through a dropout layer to reduce overfitting. Finally, it goes into a linear layer that outputs a prediction. Because BERT is pretrained on a massive corpus of text, it already understands language patterns, which can give us a high accuracy. The dropout value when the experiments started was 0.3.

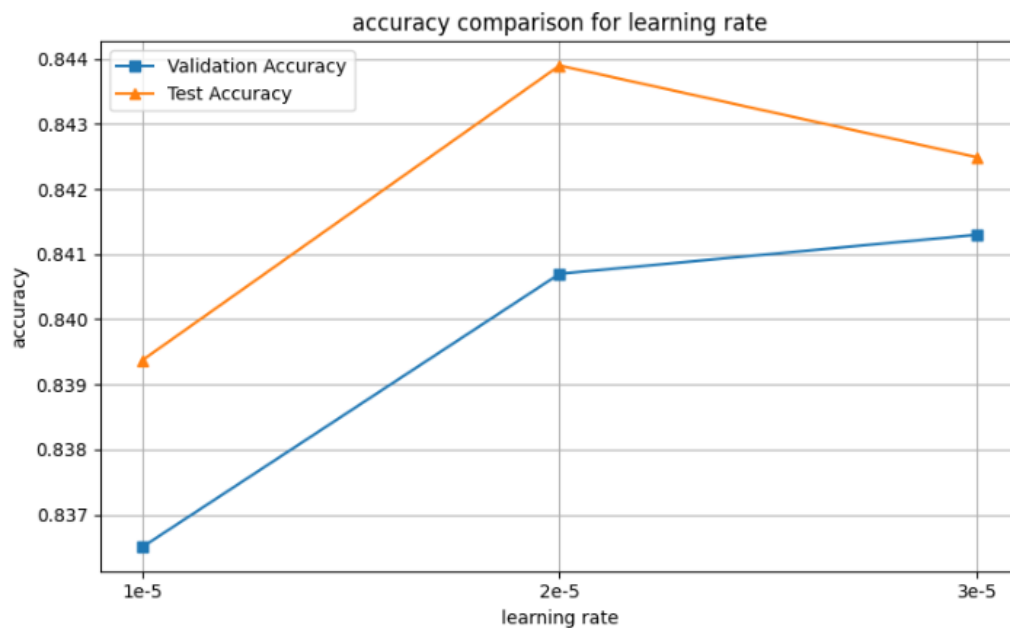
- **dropout:** Dropout is a regularization technique used in neural networks to prevent overfitting. During training, it randomly "drops out" (sets to zero) a fraction of the neurons in a layer on each forward pass. This means that the network cannot rely too heavily on any particular neuron and is forced to learn more robust, general patterns. As the plot below shows, test_accuracy is maximized with dropout=0.3 and val_accuracy is very close to the highest. For these reasons, the best value is 0.3. All the below experiments are performed with dropout=0.3.



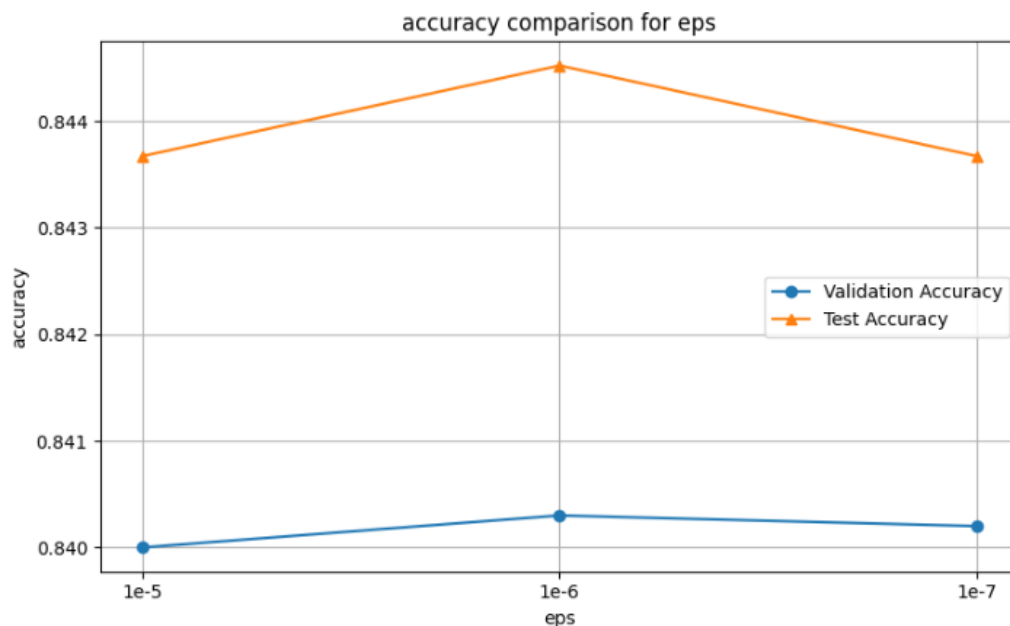
2.3.3. Optimizer. An optimizer is an algorithm that adjusts the model's weights to make the predictions better. After each batch of training data, the model makes predictions on this batch, loss is calculated between predictions and actual labels and then the optimizer uses that loss to update the model's weights so it does better next time. The hyperparameter values when the experiments started were: $lr=1e-5$ and $eps=1e-8$.

- **lr (learning rate):** The parameter learning rate controls how big is a step that the optimizer takes when it updates the model's weights during training. With

a small learning rate, careful learning is achieved. As the plot below shows, test_accuracy reaches its highest value with $lr=2e-5$ and val_accuracy is very close to the highest. For these reasons, the best value is $2e-5$. All the below experiments are performed with $lr=2e-5$.



- **eps:** The parameter eps (short for epsilon) is a very small number added for numerical stability, so optimizer avoids dividing by zero during calculations.



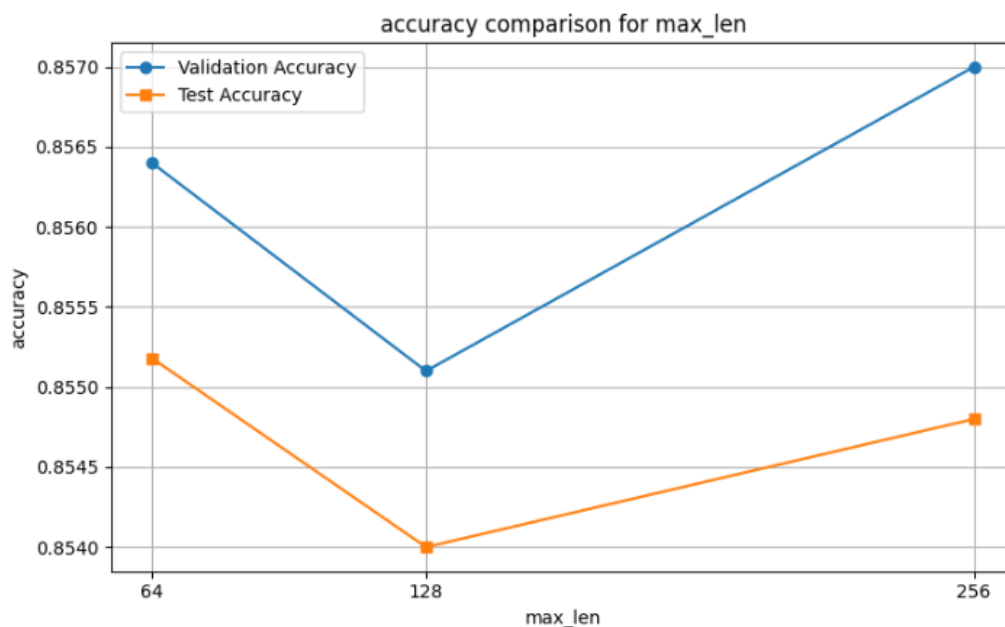
In the plot above, it can be seen that both val_accuracy and test_accuracy reach their best value with $1e-6$ as the plot below shows. For this reason, the best value for eps is $1e-6$. All the below experiments are performed with $eps=1e-6$.

2.3.4. Epochs. An epoch refers to one complete cycle through the entire training dataset by the model during the training process. During each epoch, the model processes every training example, makes predictions, calculates the error using a loss function, and updates its internal parameters (weights) to improve its accuracy. Since a single pass is

often not enough for the model to learn complex patterns, training typically involves multiple epochs. With each epoch, the model ideally becomes better at making predictions as it gradually minimizes the loss. However, using too many epochs can lead to overfitting, where the model learns the training data too well and performs poorly on new, unseen data. Before starting all the experiments, epochs were set to 4. It was observed that training for 4 epochs was time-consuming and led to overfitting, as performance on the validation set began to deteriorate. Therefore, training for 3 epochs was considered the most appropriate choice, offering a good balance between training time and model generalization.

2.3.5. BERT Dataset. The BERTDataset class prepares text data for input into a BERT model. It tokenizes each input text using `tokenizer.encode_plus`, converting it into input IDs and an attention mask, while ensuring a consistent length of `max_len` through padding and truncation. It returns a dictionary with tensors suitable for training (`input_ids`, `attention_mask`, and `labels`). This structure enables easy use with PyTorch's DataLoader for batch processing during model training and evaluation. The `max_len` value when the experiments started was 128.

- **max_len:** `max_len` is the maximum number of tokens that each input text will be padded or truncated to during tokenization. This ensures that all input sequences have the same length, which is required for efficient batch processing in BERT models. In the plot below, it can be seen that `val_accuracy` reaches its highest value with `max_len = 256`. Test accuracy is very close to the highest value, so the best value for `max_len` is 256.



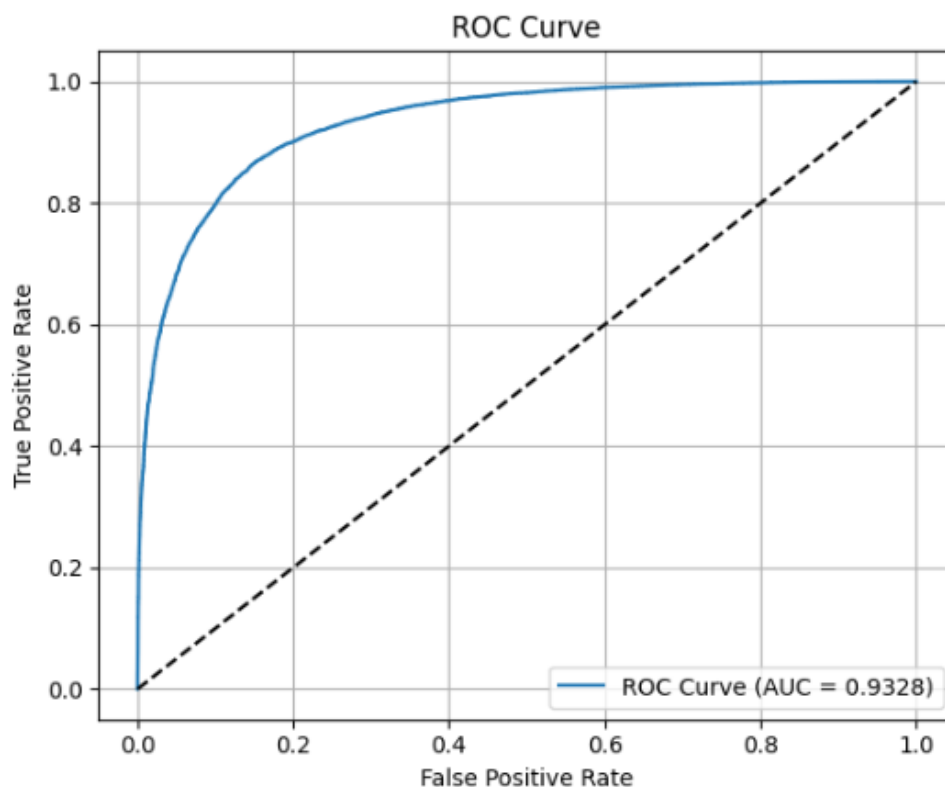
2.4. Results, plots and Overall Analysis for BERT model

The following plots were generated using the best model, which is the one with the above-selected hyperparameter values.

2.4.1. ROC curve. The ROC (Receiver Operating Characteristic) curve is a graphical representation of a classification model's performance. It shows the trade-off between the True Positive Rate and False Positive Rate.

$$\text{False Positive Rate} = \frac{\text{False Positives}}{\text{False Positives} + \text{True Negatives}}$$

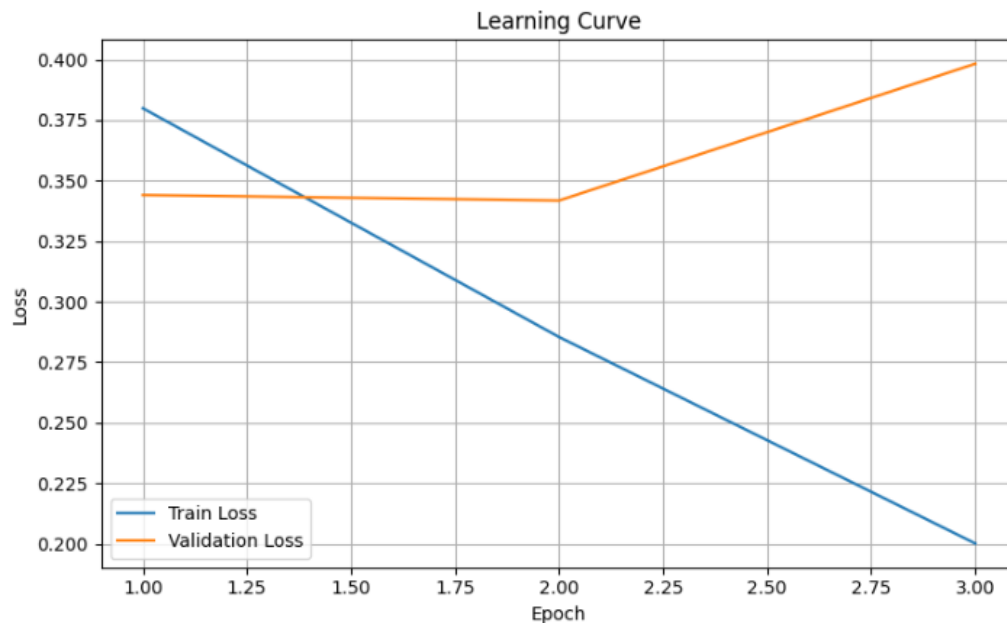
$\text{True Positive Rate} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$ The more accurate the model is, the closer the curve is to the top left corner. Below is the ROC curve for my model:



A random classifier would produce a diagonal line (AUC = 0.5). The better the model is trained, the closer the AUC gets to 1.0. It is clear that the model I trained has AUC equal to 0.9328. An AUC of 0.9328 indicates that the model has a very strong ability to distinguish between classes, which means that it correctly ranks positive instances higher than negative ones 93.28% of the time. This is considered a magnificent performance, as it is over 0.9, where models are reliable and effective. While not perfect (a perfect model would have AUC equal to 1.0), it suggests that the model makes accurate predictions and has a low error rate.

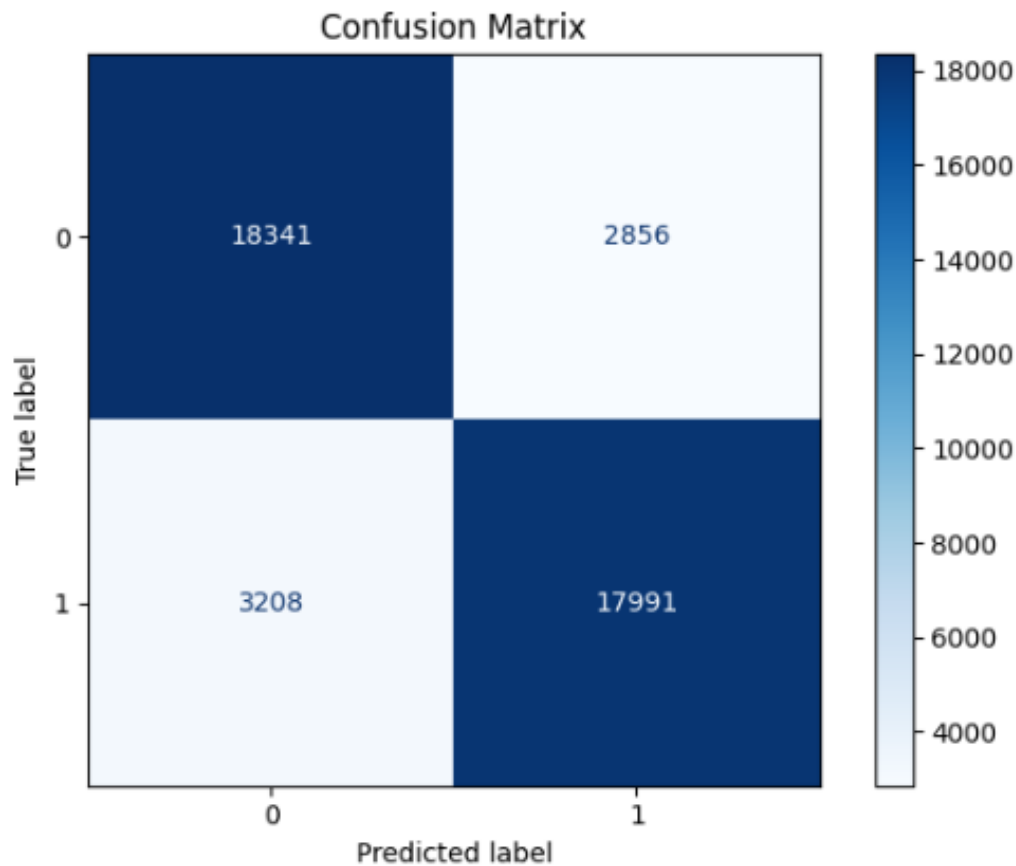
2.4.2. Learning Curve for loss. A learning curve for loss is a graph that displays how the model's loss changes over time during training and validation. It helps evaluate how well the model is optimizing and whether it is improving its ability to minimize errors. Typically, the training loss shows how well the model is fitting the training data,

while the validation loss reflects its performance on unseen data. By observing the gap and trend between the two curves, we can diagnose issues like overfitting (low training loss, high validation loss), underfitting (high loss on both), or good generalization (both losses decreasing and staying close). Below is the Learning Curve for loss in my model.



The learning curve shows that training loss consistently decreases across epochs, indicating the model is learning and fitting the training data well. However, the validation loss, despite the fact that validation loss decreases until the second epoch, it increases a lot, suggesting that while the model is improving on the training set, it is not generalizing better to unseen data. This is a sign of overfitting, where the model becomes too specialized to the training data and starts to lose the ability to perform well on new, unseen data, such as the validation set. The gap between the training and validation losses toward the end reinforces this observation.

2.4.3. Confusion matrix. The confusion matrix is a valuable tool for assessing a model's ability to make accurate predictions by highlighting its successes and errors. It compares the labels that the model predicted for the validation dataset with the correct labels.



In the plot above, there are 4 boxes.

- The top-left box shows how many tweets with label 0 the model predicted correct.
- The bottom-left box shows how many tweets with label 1 the model predicted wrong.
- The top-right box shows how many tweets with label 0 the model predicted wrong.
- The bottom-right box shows how many tweets with label 1 the model predicted correct.

It is obvious that the model predicted the correct label for $18341 + 17991 = 36332$ tweets and the wrong label for 6064 tweets. This means that the model worked with an accuracy equal to 0.8569676385, which proves that the model works properly.

2.5. Best trial

The best trial was achieved using the hyperparameter values I previously mentioned. It had the following values for the metrics:

- Validation Accuracy: 0.8569676385
- Precision: 0.8630018708
- Recall: 0.8486721072

- F1-Score: 0.8557770061

This means that the model has a good performance and it is reliable for unseen data. For the test dataset the model had an accuracy equal to 0.85480. It has a small difference with the validation accuracy (0.8570) which proves that the model can perform well on different data without overfitting.

3. DistilBERT model

3.1. Abstract

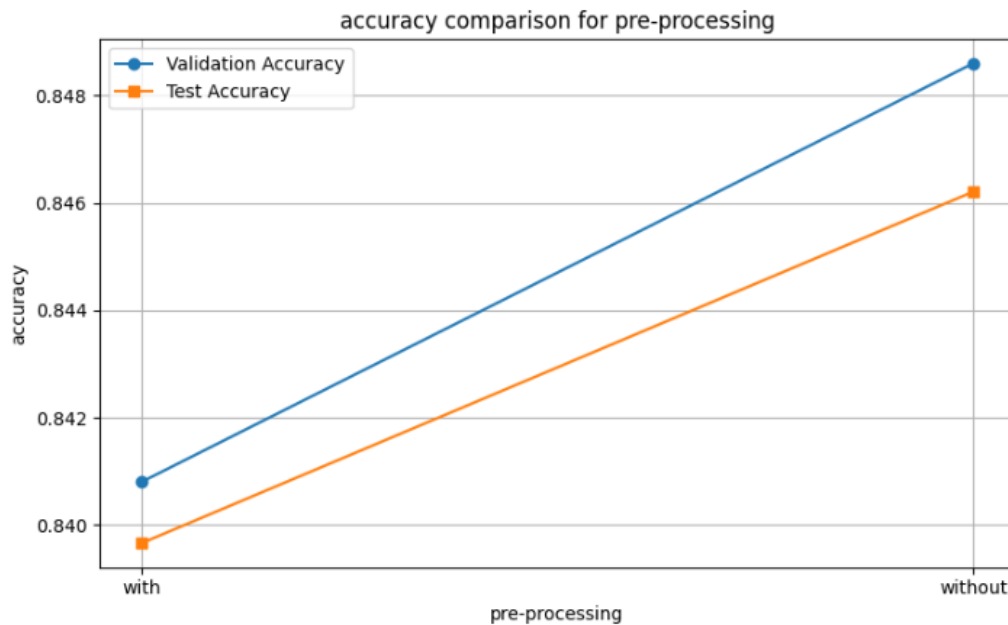
The second requirement was to address the task of sentiment classification on English-language tweets using the DistilBERT model, a lightweight variant of BERT, implemented with PyTorch and the HuggingFace Transformers library. The dataset is tokenized using DistilBERT's tokenizer, and each tweet is converted into suitable input tensors. A custom classifier is built on top of the pretrained DistilBERT model, which is fine-tuned on the labeled training data using the AdamW optimizer and a cosine learning rate scheduler with warmup. The model is trained for several epochs, with accuracy, precision, recall, and F1-score used to monitor its performance on a validation set. Regularization is applied via dropout, and the model with the best validation accuracy is saved. After training, I evaluate the final model using key metrics and learning curves, and use it to generate sentiment predictions on a test dataset. Random seeds are consistently set throughout the pipeline to ensure the results are reproducible.

3.2. Data processing and analysis for distilBERT model

3.2.1. Pre-processing. I tried the following pre-processing techniques:

1. Conversion of all uppercase letters to lowercase.
2. Removal of all whitespaces.
3. Replace all possible contractions with their normal form("don't" -> "do not").
4. Replacement of "&";" with & and """ with ".
5. Replacement of all emojis with ".
6. Removal of all non-ASCII characters.
7. Replacement all mentions starting with "@" with the word "mention".
8. Replacement all mentions starting with "http" with the word "url".
9. Removal of all punctuation.
10. Removal of all whitespaces again to make sure that no whitespace is left.

The plot below shows the accuracy of the initial model, **which has the same hyperparameter values with the best BERT model**, with and without pre-processing:



It is clear that the model performs better without pre-processing. This happens because distilBERT is pretrained on raw, unprocessed text data. The model has already learned to handle natural variations in language, including punctuation, casing, and common stop words. Applying pre-processing can actually strip away valuable contextual information that the model relies on to understand the nuances of meaning. By preserving the original structure and tokens of the input, DistilBERT can better leverage its pretrained knowledge, leading to improved performance.

3.2.2. Data partitioning for train, test and validation for distilBERT model. As in the task for BERT model, dataset was already partitioned into 3 files (train, val and test).

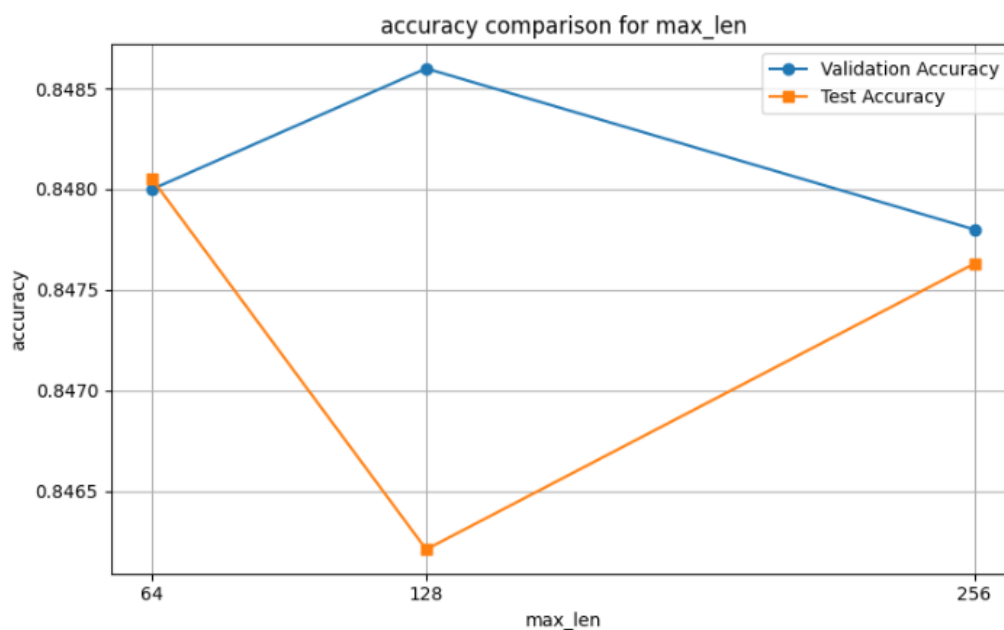
- **Training dataset:** This data is used for the model training. The model learns from it so that it can predict labels to other datasets later.
- **Validation dataset:** After each epoch, the model is evaluated on the validation set. It is used to tune hyperparameters and conclude to the best version of the model.
- **Test dataset:** The model predicts labels on data that the model has never seen before and gives the final estimate of its true performance.

3.2.3. Vectorization. DistilBERT follows a vectorization process that is largely identical to BERT, utilizing the same tokenization strategy and input formatting techniques. Specifically, the distilBERT tokenizer converts raw text into subword token IDs based on a fixed vocabulary. During this process, special tokens like [CLS] and [SEP] are added, sequences are padded or truncated to a predefined maximum length, and attention masks are generated to distinguish actual tokens from padding. These token IDs and attention masks are then passed into the DistilBERT model, which produces contextualized embeddings for each token. Despite being a lighter and faster version of BERT, DistilBERT retains the ability to generate rich semantic representations suitable for tasks such as sentiment classification.

3.3. Experiments and hyper-parameter tuning for distilBERT model

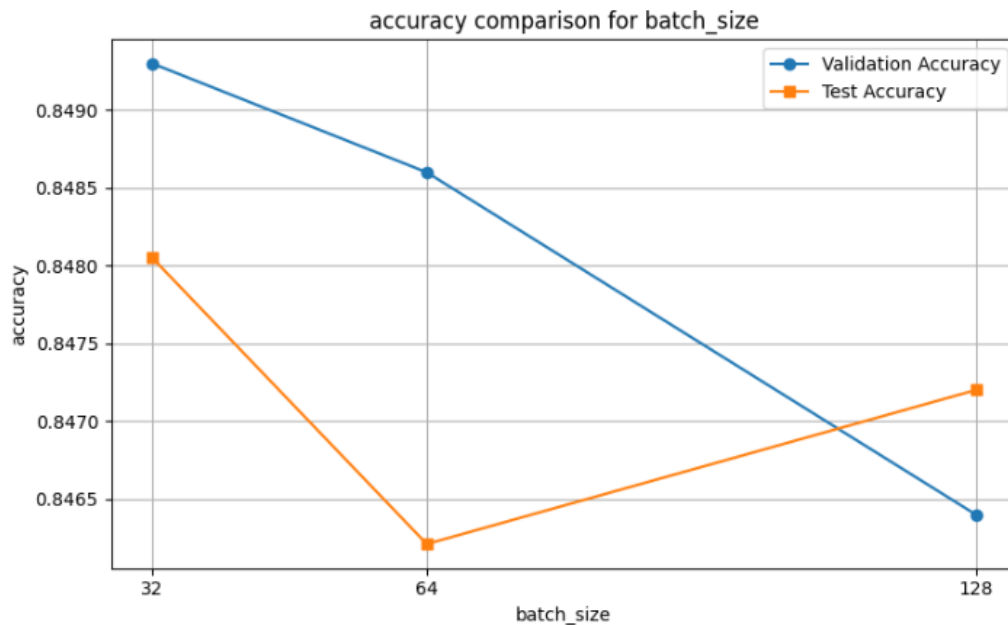
3.3.1. DistilBERT Dataset. The DistilBERT Dataset class tokenizes input text using the DistilBERT tokenizer, converting it into token IDs and attention masks with a fixed maximum length (`max_len`) through padding and truncation. It prepares the data in tensor format, including labels if provided, making it ready for training or evaluation. When experiments started for the distilBERT model, `max_len` was 128.

- **max_len:** In the plot below, it can be seen that the best `val_accuracy` is reached with value 128. `Test_accuracy` is not the best, but validation accuracy is more important during model development because it guides model selection and tuning. It reflects how well the model performs on unseen data that was not used for training, helping detect overfitting. Test accuracy, on the other hand, should be used only once, after finalizing the model, to report its performance. Relying on test accuracy during development risks leaking information and producing overly optimistic results. For this reason, best value for `max_len` is 128. All further experiments are performed with `max_len=128`.



3.3.2. DataLoader. The DataLoader for DistilBERT batches and serves tokenized input data to the model during training and evaluation. It wraps the custom DistilBERT-Dataset, which handles tokenization, adds special tokens, truncates inputs to a fixed `max_len`, and creates attention masks. The DataLoader ensures efficient data loading, shuffling (during training), and consistent batching, making it essential for feeding the model correctly formatted inputs. The hyperparameter values when the experiments started were: `batch_size=64`, `shuffle=True` (only for train dataset) and `generator=g` (only for train dataset).

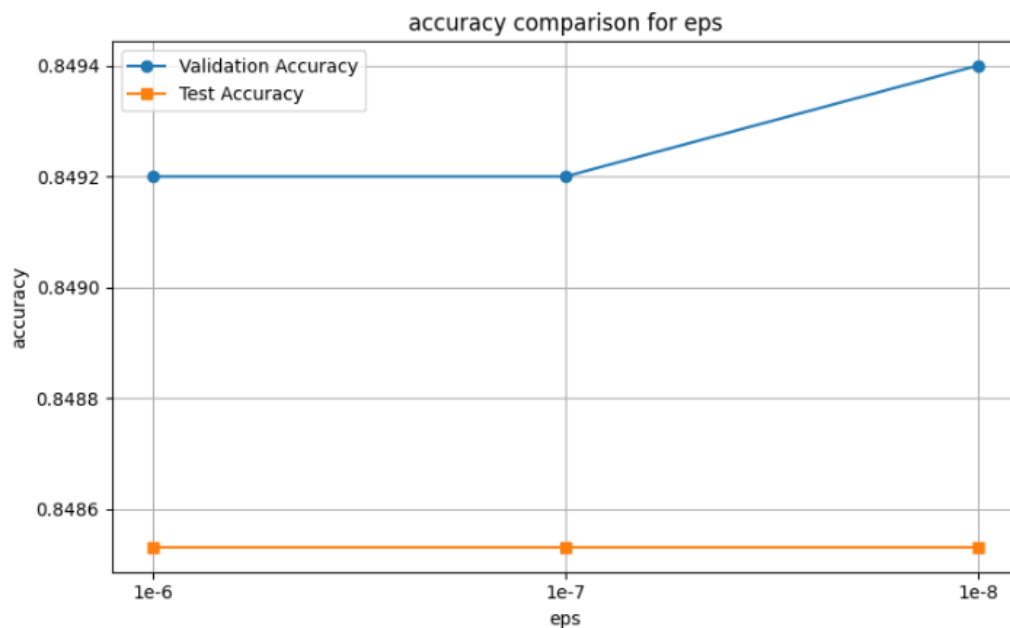
- **batch_size:** As the plot below shows, both val and test accuracy reach their highest value with `batch_size=32`. A batch size of 32 provides a good balance between learning and generalization while larger batch sizes may lead to overfitting and poor generalization. For these reasons, the best value is 32. All further experiments are performed with `batch_size=32`.



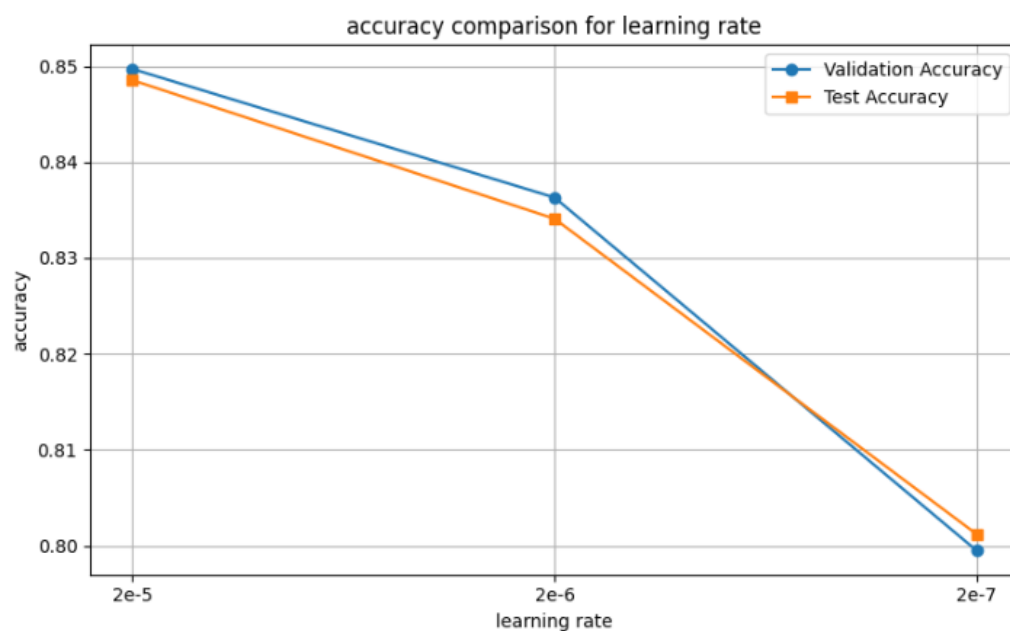
- **shuffle:** For DataLoader of validation dataset, shuffle must be False, because the model will predict labels and the tweets cannot be shuffled. For the training dataset, shuffle must be True because shuffling improves generalization and avoids overfitting.
- **generator:** By passing a `torch.Generator()` with a fixed seed(42), it is ensured that shuffling is deterministic and reproducible.

3.3.3. Optimizer. The AdamW optimizer is used to train the DistilBERT classifier. AdamW is a variant of the Adam optimizer that includes weight decay, which helps improve generalization and reduce overfitting. During training, the optimizer updates the model's parameters based on the gradients computed from the loss function, gradually improving the model's performance by minimizing prediction errors. The hyperparameter values when the experiments started were: `lr=2e-5`, `eps=1e-6`.

- **eps:** As it can be seen in the plot below, test_accuracy remains the same for all the experiments for eps but val_accuracy reaches its highest value with eps=1e-8. For this reason, the value chosen is 1e-8. All further experiments are performed with this value.



- **lr (learning rate):** The plot below shows that the model is more accurate with a bigger learning rate. The best validation and test accuracies were achieved with a learning rate of 2e-5, while smaller values like 2e-6 and 2e-7 led to lower results. This is because very small learning rates cause the model to update its weights too slowly, preventing it from learning meaningful patterns within a limited number of epochs. A moderate learning rate like 2e-5 strikes a good balance, allowing the model to converge efficiently. For this reason, the value chosen is 2e-5. All further experiments are performed with 2e-5.

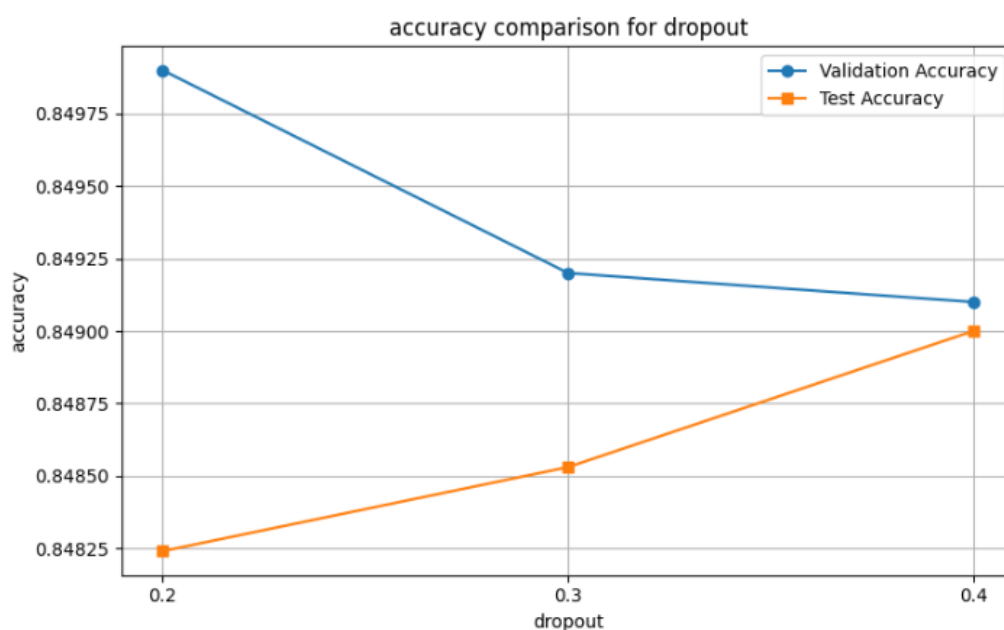


- **weight_decay:** The parameter weight_decay is a form of regularization. It helps the model prevent from overfitting by discouraging it from relying too heavily

on large weight values. It helps the model generalize better and not memorize the data too closely.

3.3.4. DistilBERT Classifier. The DistilBERTClassifier is a neural network model that uses a pretrained DistilBERT transformer to extract contextual representations from input text. It takes tokenized text, passes it through DistilBERT, and uses the [CLS] token's embedding to represent the whole sequence. This embedding is then passed through a dropout layer and a linear classifier to predict the sentiment label (positive or negative).

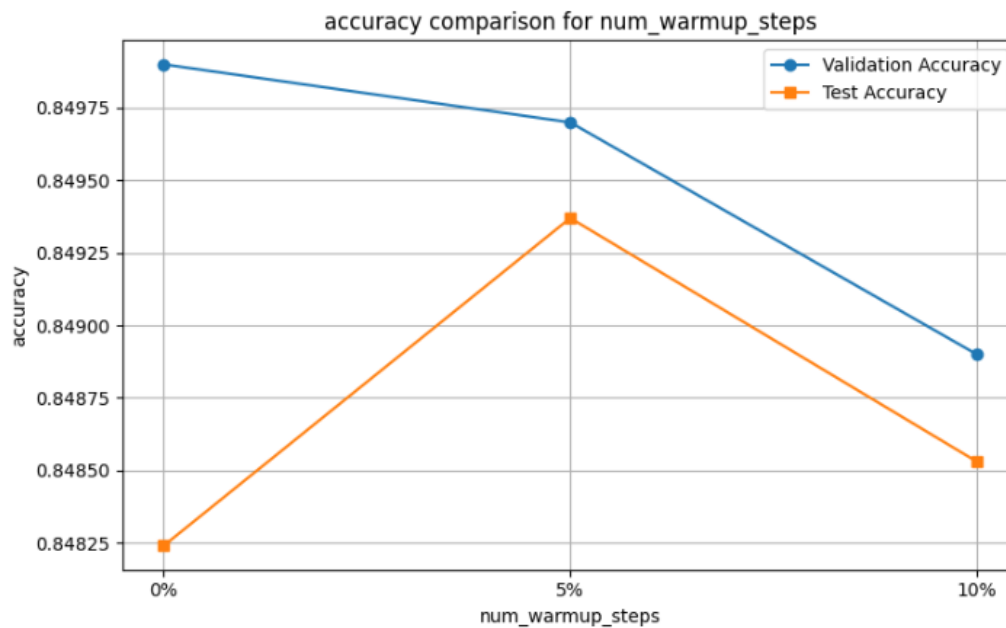
- **dropout:** In the plot below, it can be seen that val_accuracy has its highest value with dropout=0.2 while test_accuracy has its highest value with dropout=0.4. However, val_accuracy is more important as explained above. So the value chosen is 0.2. All further experiments are performed with dropout=0.2.



3.3.5. Scheduler. The scheduler adjusts the learning rate during training to improve performance. It begins with a warmup phase where the learning rate gradually increases, helping stabilize early training. After that, it decreases the learning rate using a cosine schedule, allowing the model to make smaller, more precise updates. This approach helps the model converge more effectively and avoid issues like overshooting or unstable training.

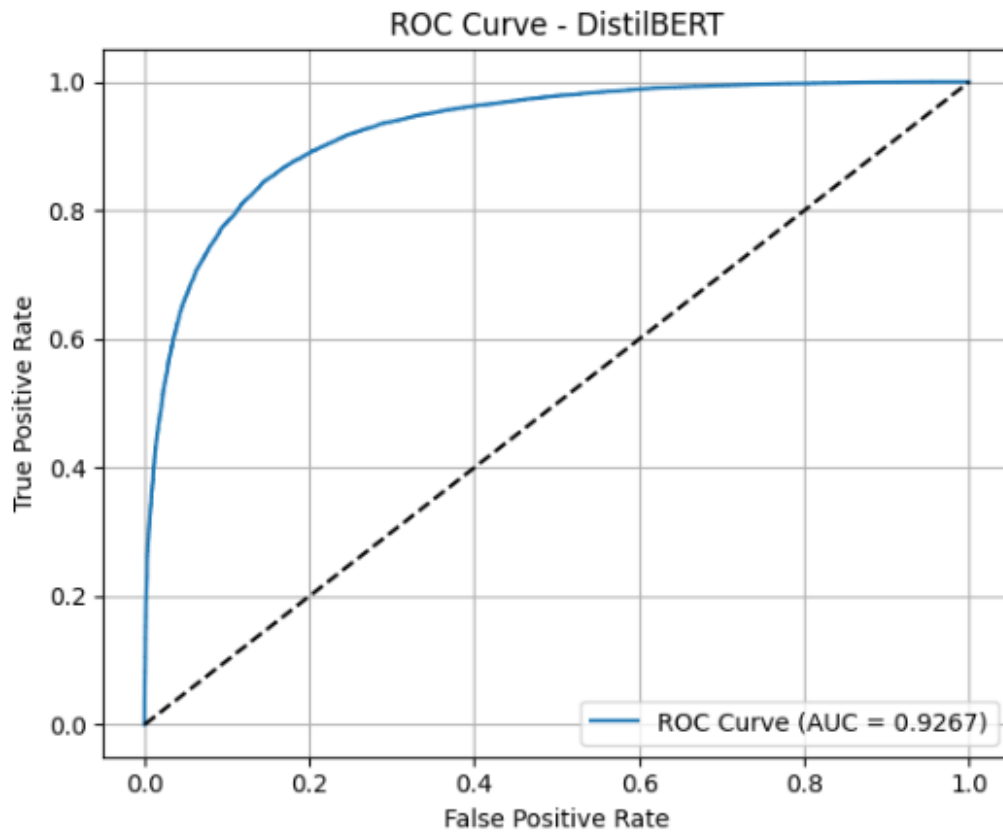
- **num_warmup_steps:** num_warmup_steps defines how many training steps are used to gradually increase the learning rate from a very small value to the target learning rate at the beginning of training. This warm-up phase helps stabilize early training, preventing the model from making large, unstable updates before it starts learning meaningful patterns. In the plot below, val and test_accuracy values can be shown with different num_warmup_steps (5% of total_steps and 10% of total_steps). As the plot below shows, val_accuracy reaches its highest value with num_warmup_steps=0 while test_accuracy is maximized with num_warmup_steps=0.05 * total_steps. However, the difference of val_accuracy

between these two cases is insignificant. For this reason, `num_warmup_steps` is 5% of the amount of `totsal_steps`.

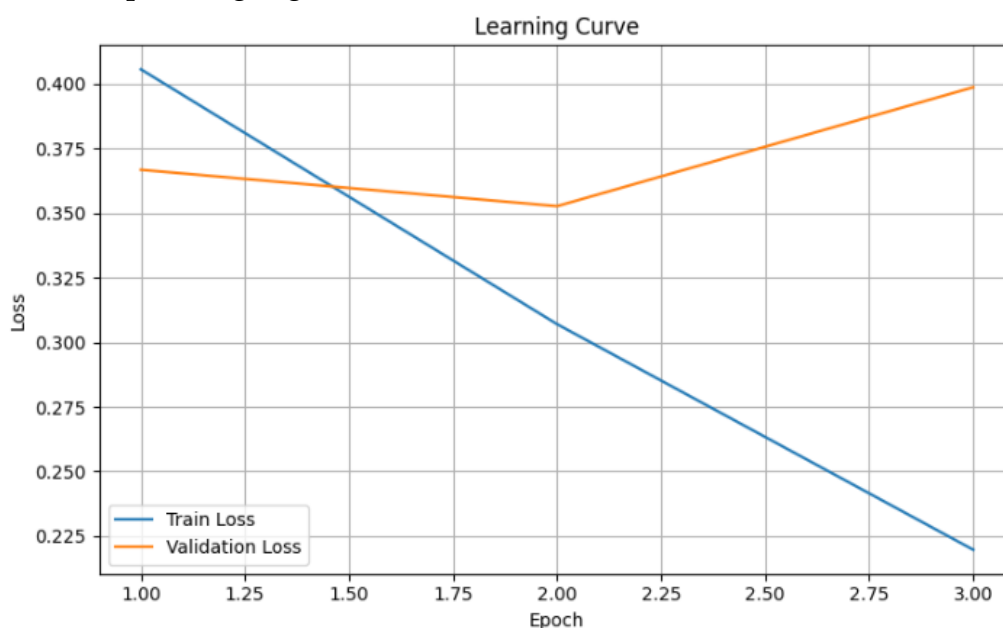


3.4. Results, plots, and Overall Analysis for DistilBERT model

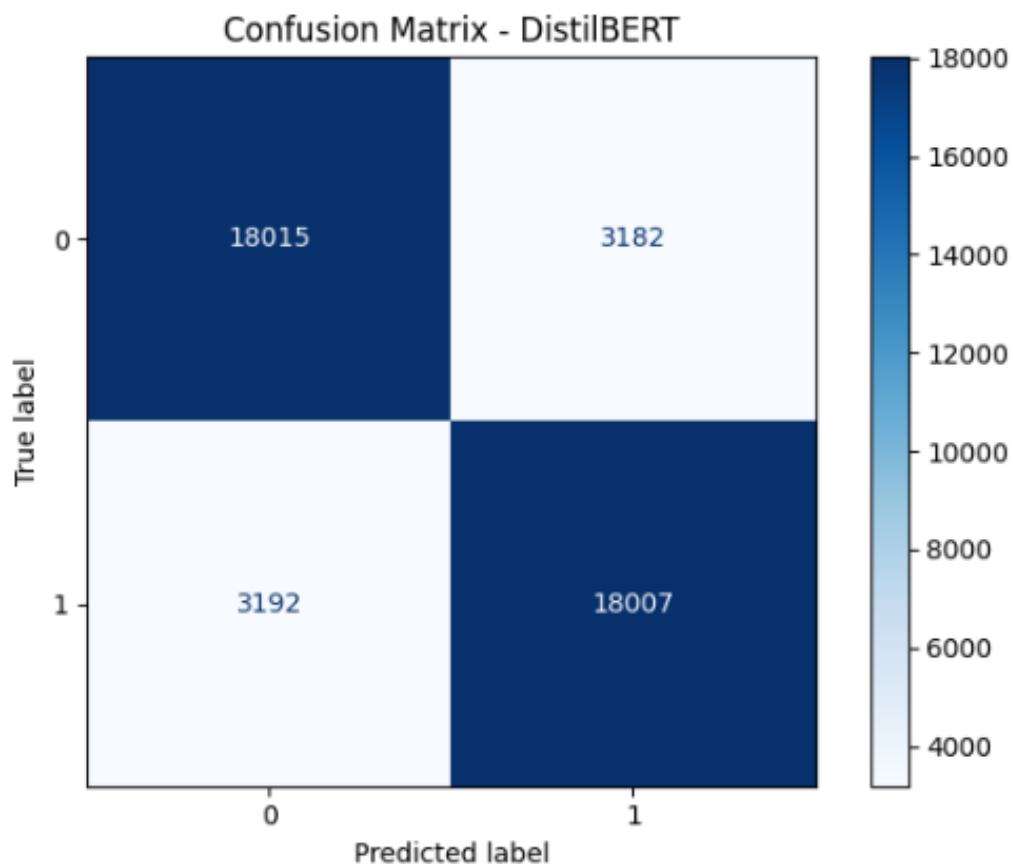
3.4.1. ROC Curve. The ROC curve for the DistilBERT classifier indicates that the model performs very well in distinguishing between the positive and negative classes. With an AUC (Area Under the Curve) of 0.9267, the model achieves a 92.67% probability of correctly ranking a randomly chosen positive instance higher than a negative one. This is a strong result, as AUC values above 0.9 generally reflect excellent model performance. The curve's shape — rising sharply toward the top-left corner — shows that the model maintains a high true positive rate while keeping the false positive rate low. In practical terms, this means DistilBERT makes few classification errors and is highly reliable for binary prediction tasks.



3.4.2. Learning Curve for loss. The learning curve below, illustrates the behavior of loss for the DistilBERT model during training over three epochs. Training loss consistently decreases, which indicates that the model is effectively learning from the training data. However, validation loss initially decreases slightly but, after the second epoch, it starts to rise indicating that overfitting is beginning to occur. This means the model continues to improve on the training data, but its ability to generalize to unseen validation data deteriorates beyond epoch 2. The gap between training and validation loss in the final epoch highlights this issue.



3.4.3. Confusion Matrix. This confusion matrix below, illustrates the performance of the DistilBERT classifier on a binary classification task. Out of all true class 0 instances, 18,015 were correctly predicted (true negatives), while 3,182 were incorrectly classified as class 1 (false positives). Similarly, 18,007 class 1 instances were correctly identified (true positives), and 3,192 were wrongly predicted as class 0 (false negatives). The model shows great performance as the numbers of true positives and true negatives are high (with accuracy equal to 0.8496556279) and the number of misclassifications relatively low. This indicates that the classifier is effectively distinguishing between the two classes with consistent accuracy.



3.5. Best trial

The best trial was achieved using the hyperparameter values I previously mentioned. It had the following values for the metrics:

- Validation Accuracy: 0.8496556279
- Precision: 0.8498277408
- Recall: 0.8494268598
- F1-Score: 0.8496272530

This indicates that the model performs well and is dependable when applied to new, unseen data. On the test set, it achieved an accuracy of 0.84937, which is very close to the validation accuracy of 0.8497. The minimal gap between these scores suggests that the model generalizes effectively and is reliable.

4. Comparison of BERT and distilBERT model

Undoubtedly both models demonstrate strong performance, delivering high-quality results across all evaluation metrics. BERT, known for its strong language understanding capabilities due to its larger architecture and deeper layers, achieved slightly better performance across most evaluation metrics. Specifically, it reached a validation accuracy of 0.8569, with a precision of 0.8630, recall of 0.8486, and an F1 score of 0.8557. DistilBERT, which is a distilled and more compact version of BERT, delivered a validation accuracy of 0.8496, precision of 0.8498, recall of 0.8494, and F1 score of 0.8496. These results highlight that while BERT slightly outperforms DistilBERT in precision and overall accuracy, the differences are relatively minor.

The trade-off between the two models becomes more evident when considering computational efficiency. DistilBERT is approximately 40% smaller and 60% faster than BERT, while retaining about 97% of its language understanding capabilities. The nearly identical recall values (BERT: 0.8487, DistilBERT: 0.8494) suggest that both models are similarly capable of identifying relevant instances. However, BERT's higher precision indicates it makes fewer false-positive predictions. The F1 scores, which balance precision and recall, further confirm BERT's slight advantage in predictive reliability.

Overall, BERT is more accurate, but its higher computational cost makes it less suitable for resource-constrained or real-time applications. DistilBERT, on the other hand, offers a compelling balance between performance and efficiency, making it a practical choice when model size and speed are critical factors. Thus, the selection between these two models should be guided by the specific requirements of the task, including performance thresholds and deployment constraints.

5. Comparison with TF-IDF + Logistic Regression model

A comparison of the three models used in this project clearly shows that transformer-based models (BERT and distilBERT) consistently surpass the traditional method across all major evaluation metrics. The TF-IDF + Logistic Regression model achieved a validation accuracy of approximately 80.52%, with precision, recall, and F1-score all around 80.52% as well. This reflects a consistent and balanced performance, yet it is clearly outmatched by both transformer models. DistilBERT, a lighter and faster version of BERT, yielded a validation accuracy of 84.96%, along with precision, recall, and F1-score values also centered around 84.96%. Despite its reduced size, DistilBERT captures contextual meaning effectively, providing strong performance with lower computational cost. However, the BERT model surpassed all others, achieving the highest validation accuracy of 85.69%, with precision at 86.30%, recall at 84.86%, and F1-score at 85.57%. These metrics indicate that BERT not only identifies more true positives (higher recall) but also maintains high precision, making fewer incorrect predictions. Overall, while TF-IDF remains a simple and interpretable method, both DistilBERT and BERT leverage deep contextual embeddings to deliver more nuanced and accurate sentiment classification, with BERT proving to be the most robust model at the cost of higher resource usage.

6. Comparison with Word2Vec + Neural Network model

When comparing the Word2Vec-based neural network to the transformer models BERT and DistilBERT, it's clear that the transformer models (BERT and distilBERT) perform better in all important evaluation metrics. The Word2Vec model, even though it used well-tuned hyperparameters and a solid neural network design, reached a validation accuracy, precision, recall, and F1-score of 78.83%. On the other hand, DistilBERT achieved 84.96% accuracy, 84.98% precision, 84.94% recall, and 84.96% F1 score. BERT performed even better, with 85.69% accuracy, 86.30% precision, 84.86% recall, and an F1 score of 85.57%.

These results are expected because of how the models are built. Word2Vec uses fixed word embeddings and only captures nearby word relationships, which limits its understanding of context. BERT and DistilBERT, however, use attention mechanisms that let them consider the full context of each word in a sentence. This leads to more meaningful and flexible representations. Even though DistilBERT is a smaller, faster version of BERT, it still keeps most of its performance. Overall, transformer models are much better at handling complex language tasks like sentiment classification than older methods like Word2Vec.